



Week 6 Workshop

Language Models

IFN647

School of Computer Science
Queensland University of Technology

Learning Objectives

- Understand inverted index construction (covered in last week's workshop)
- Apply language model scoring
- Implement Jelinek-Mercer smoothing

Tasks

➤ Task 1:

Build an inverted index from a folder of documents using `index_docs()`. (covered in last week's workshop)

➤ Task 2:

Compute document relevance scores for a query using maximum likelihood estimation. (Reference program available in the lecture notes)

➤ Task 3:

Apply Jelinek-Mercer smoothing to improve retrieval with collection-level statistics.

➤ Task 4:

Test and compare your retrieval functions using provided document sets and queries.

Knowledge Hints: Language Models

A language model is a **probability distribution over words** that tells us how likely a word is to appear in a given context.

- Example: If a document collection contains only 5 words:

`{w1, w2, w3, w4, w5}`, a language model might assign:

`P(w1) = 0.2, P(w2) = 0.1, P(w3) = 0.35, P(w4) = 0.25, P(w5) = 0.1`

- It can be used to **generate sentences** or measure relevance between documents and queries.

Language Models in Information Retrieval (IR)

- In IR, language models estimate how likely a document is to “generate” a query.

- Example:

Query: `{econom, espionag}`

Document A: `{US, econom, spy}`

→ Compute: $P(Q|D_A) = P(\text{econom} | A) \times P(\text{espionag} | A)$

- This probability helps us **rank documents based on relevance**.

- **Raw likelihood** (Query-likelihood), **Smoothed likelihood** with **Jelinek-Mercer** (JM Smoothing)

Knowledge Hints: Query-Likelihood Model

- In IR, language models estimate **how likely a document is to generate a query**.
- This approach is known as the **Query Likelihood Model**.

$$P(Q|D) = \prod_{i=1}^n P(q_i|D) \quad \text{where,}$$

$$P(q_i|D) = \frac{f_{q_i,D}}{|D|}$$

- $f_{q_i,D}$: frequency of term q_i in document D
- $|D|$: total number of terms in document D

Query: $Q = \{\text{econom, spy}\}$

Document A: $D = \{\text{US, econom, spy}\}$

- $|D| = 3$, $f(\text{"econom"}, D) = 1$, $f(\text{"spy"}, D) = 1$
- $P(Q|D) = \frac{1}{3} \times \frac{1}{3} = \frac{1}{9} = 0.111$

Query: $Q = \{\text{econom, spy, espionag, market}\}$

Document B: $D = \{\text{econom, spy, espionag}\}$

- $|D| = 3$, $f(\text{"market"}, D) = 0$
- $P(Q|D) = \frac{1}{3} \times \frac{1}{3} \times \frac{1}{3} \times 0 = 0$

Query: $Q = \{\text{econom, spy, espionag, market}\}$

Document C: $D = \{\text{econom}\}$

- $|D| = 1$, $f(\text{"spy"}, D) = 0$, $f(\text{"espionag"}, D) = 0$, $f(\text{"market"}, D) = 0$
- $P(Q|D) = \frac{1}{1} \times 0 \times 0 \times 0 = 0$

- If query words are missing from document, score will be zero
 - This is the major problem.
 - E.g., missing 1 out of 4 query words same as missing 3 out of 4.
 - This is clearly not appropriate for longer queries.

Knowledge Hints: Jelinek-Mercer Smoothing

◆ Why Smoothing?

- In the query likelihood method, missing query terms make $P(Q|D) = 0$, even if the document is mostly relevant.
- **Smoothing assigns some probability to unseen terms** by based on the overall collection.

$$P(Q|D) = \prod_{i=1}^n P(q_i|D) \quad \text{where,} \quad p(q_i|D) = (1 - \lambda) \frac{f_{q_i,D}}{|D|} + \lambda \frac{c_{q_i}}{|C|}$$

- $f_{q_i,D}$: frequency of term q_i in document D
- c_{q_i} : frequency of term q_i in the collection C
- $|D|$: total number of terms in D
- $|C|$: total number of terms in the collection C
- λ : smoothing parameter (e.g., 0.4)

◆ Example (Using $\lambda = 0.5$)

Query: $Q = \{\text{US, econom, espionag}\}$

Document D : $\{\text{US, econom, spy}\}$

Collection C : $\{\text{D1, D2, D3, D4, D5, D6}\}$

- $P(\text{US}|D) = 0.5 \cdot \frac{1}{3} + 0.5 \cdot \frac{4}{25} = 0.166 + 0.08 = 0.246$
- $P(\text{econom}|D) = 0.5 \cdot \frac{1}{3} + 0.5 \cdot \frac{3}{25} = 0.166 + 0.06 = 0.226$
- $P(\text{espionag}|D) = 0.5 \cdot 0 + 0.5 \cdot \frac{3}{25} = 0 + 0.06 = 0.06$
- $P(Q|D) = 0.246 \times 0.226 \times 0.06 = 0.0033$

✓ Benefit:

- The result is non-zero even when some query terms are missing from the document.
- This allows for **fairer document ranking** in real-world search systems.

Task1

covered in last week's workshop

Task 1. Design function `index_docs()` to construct an inverted index, a dictionary

```
{term:{docID1:freq1, DocID2:freq2, ...}, ...}.
```

For each document in the "Test_Doc" folder, it firstly finds the "docid" in tag <newsitem>. For each index term (excluding number, punctuations, and tags <p> and </p>) in <text>, it aslo inserts the term into the index or accumulates its frequency.

Task 2

Task 2: Let $Q = \{q_1:1, q_2:1, \dots, q_n:1\}$ be a dictionary, please define a function `likelihood_IR(I, Q)` to estimate $P(Q|D)$, i.e., it returns the score of document D for the given query Q by using

$$P(Q|D) = \prod_{i=1}^n P(q_i|D)$$

$$P(q_i|D) = \frac{f_{q_i,D}}{|D|}$$

where $f_{q_i,D}$ is the number of times word q_i occurs in document D , and $|D|$ is the number of words in D .

`I`

`Index = index_docs(data_path, stop_words)`

`likelihood_IR(I, Q)`

Input:

Go to Task4 to know about inputs:

Test the defined functions using the provided set of test documents (in folder "Test_docs").

You need to use the 'common-Englishwords.txt' when indexing both sets of documents into Index_Test.

you can assume `lamda = 0.4`, and `Query = {'formula':1, 'one':1}, {'share':1}` or `{ 'leaderboard':1, 'british':1}`.

Output for Query2, {'share':1}

Likelihood Query Result -----

Document ID: 807606 and relevance weight: 0.017937219730941704

Document ID: 809481 and relevance weight: 0.011299435028248588

Document ID: 807600 and relevance weight: 0.003179650238473768

Task 3

Task 3: Assume you have obtained another folder, e.g., "Collection_C". Extend function `likelihood_IR(I, Q)` to a Jelinek-Mercer smoothing function `likelihood_JM(I_C, I_D, Q, lambda)` by using the following equation:

$$P(Q|D) = \prod_{i=1}^n \left((1 - \lambda) \frac{f_{q_i, D}}{|D|} + \lambda \frac{c_{q_i}}{|C|} \right)$$

where c_{q_i} is the number of times query term q_i occurs in I_C , and $|C|$ is the total number of term occurrences in I_C . See more details in page 12 of week 5 lecture notes.

The function estimates a score for documents in I_D .

`I_D`

`Index = index_docs(data_path, stop_words)`

`I_C`

`Index = index_docs(data_path, stop_words)`

`likelihood_JM(I_C, I_D, Q, lambda)`

Input:

Go to Task4 to know about inputs:

Test the defined functions using the provided set of test documents (in folder "Test_docs") and a collection of documents (in folder "collection_C").

You need to use the 'common-Englishwords.txt' when indexing both sets of documents into Index_Test.

you can assume `lambda = 0.4`, and `Query = {'formula':1, 'one':1}, {'share':1}` or `{ 'leaderboard':1, 'british':1}`.

Output for Query2, {'share':1}

```
JM Smoothing-Based Query Result -----
Document ID: 807606 and relevance weight: 0.012240164350880293
Document ID: 809481 and relevance weight: 0.008257493529264423
Document ID: 807600 and relevance weight: 0.003385622655399532
Document ID: 809495 and relevance weight: 0.0014778325123152713
```


Task 4: Solution → refer Jupyter notebook

Task 4: Test the defined functions using the provided set of test documents (in folder "Test_docs") and a collection of documents (in folder "collection_C"). You need to use the 'common-english-words.txt' when indexing both sets of documents into Index_Test and Index_C, respectively. You can then test functions likelihood_IR(Index_Test, Query) and likelihood_JM(Index_C, Index_Test, Query, lamda) (you can assume lamda = 0.4, and Query = {'formula':1, 'one':1}, {'share':1} or {'leaderboard':1, 'british':1}).